DBMS • OAVS COMPUTER TEACHER (TGT) PREPARATION

Chapter 9

Recovery Mechanisms

TOPICS COVERED

1. Recovery from problems of volatile & non-volatile storage devices
2. Concept, properties and states of a Transaction
3. Log-based Recovery
4. Checkpoint
5. Shadow Paging

Chapter 9: Recovery Mechanisms

A complete, exam-oriented study note — from basics to advanced exam-level detail

> Database **recovery** is the process of restoring the database to a consistent state after a failure (power failure, system crash, disk crash, or transaction error). This chapter explains *why* failures cause problems, *what* a transaction is, and the three classical recovery mechanisms used by every DBMS: **log-based recovery**, **checkpointing**, and **shadow paging**.

9.1 Recovery from Problems of Volatile & Non-Volatile Storage Devices

1.1 Why Storage Type Matters for Recovery

Every recovery technique is built around one question: **"Which part of the storage system can we trust after a failure, and which part might be lost or corrupted?"** To answer this, storage is classified into three categories based on how failures affect the data stored in them.

(a) Volatile Storage

Volatile storage **loses its contents when power is lost**. Examples: main memory (RAM), cache memory, and CPU registers. Volatile storage is fast and directly accessible by the CPU, but it is unreliable for permanent storage because a simple power cut wipes it out completely.

Problem: If a transaction has updated data, only in the database buffers (RAM) and the system crashes before the data is written to disk, all those updates are lost.

(b) Non-Volatile Storage

Non-volatile storage **retains data even after power loss**. Examples: magnetic disks, SSDs, flash memory, magnetic tapes. This is where the actual database resides permanently.

Problem: Non-volatile storage is not 100% failure-proof either. It can suffer from **disk crashes / head crashes**, where the read/write head touches the disk surface and destroys some or all of the data on it. Such failures are rare but far more serious because the loss can be permanent.

(c) Stable Storage

Stable storage is a **theoretical/idealised form of storage that never loses data**, not even during a failure while a write is in progress. In practice, true stable storage cannot exist, but it is **approximated** by:

- **Mirroring** – maintaining multiple identical copies of data on physically separate disks (RAID).
- **Sequential writes with checks** – writing to one copy, verifying it, then writing to the next copy, so that at least one consistent copy always survives a crash during the write itself.

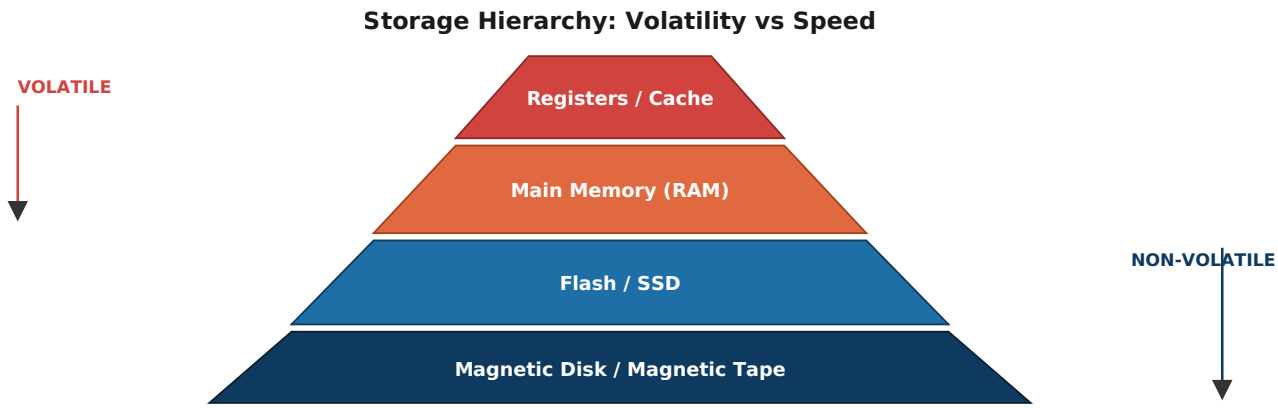


Fig 9.1 — Storage hierarchy showing volatile storage (fast, erased on power loss) at the top and non-volatile storage (permanent, slower) below.

1.2 Types of Failures Recovery Must Handle

Table 9.1 — Classification of Failures

Failure Type	Description	Storage Affected
Transaction failure	Logical error (bad input, integrity violation) or system error (deadlock) causes a transaction to stop before completion.	None lost — only in-progress transaction affected.
System crash	Hardware/software bug, OS crash, or power failure halts the system; contents of main memory are lost.	Volatile storage (RAM) contents lost; disk assumed safe (no data corruption).
Disk failure	Head crash or media failure destroys part or all of the data stored on disk.	Non-volatile storage; needs backup/ archival copy for recovery.

EXAM POINT

Recovery techniques are designed mainly to counter the effects of **volatile storage loss** during a system crash — this is exactly why the **log** (kept on stable/non-volatile storage) is central to almost every recovery scheme.

REMEMBER

Volatile → RAM, cache, registers (data lost on power failure).
Non-volatile → disk, SSD, tape (data survives power failure but can suffer physical damage).
Stable storage → an idealised, failure-proof storage approximated using mirrored/redundant disks.

COMMON MISTAKE

Students often think “stable storage” is a real, separate hardware device. It is **not** — it is a concept implemented using ordinary non-volatile storage with redundancy (mirroring) to make data loss extremely unlikely.

9.2 Concept, Properties and States of Transaction

2.1 What is a Transaction?

A **transaction** is a single logical unit of work that accesses and possibly updates the data items of a database. It is a sequence of one or more operations (read/write) that must be executed **as a whole** — either completely or not at all. Example: transferring money from Account A to Account B involves two write operations, but both must succeed or both must fail together.

```
T1: read(A);
    A := A - 500;
    write(A);
    read(B);
    B := B + 500;
    write(B);
    commit;   // classic bank fund-transfer transaction
```

2.2 ACID Properties of a Transaction

ACID is the set of four properties that guarantee a database transaction is processed reliably. It stands for: **A** — Atomicity, **C** — Consistency, **I** — Isolation, **D** — Durability.

Table 9.2 — ACID Properties

Property	Meaning	Recovery Manager's Role
Atomicity	A transaction is treated as a single, indivisible unit — either *all* its operations are reflected in the database, or *none* are.	Undo partial effects of a failed transaction (rollback).
Consistency	Execution of a transaction, in isolation, takes the database from one consistent state to another.	Ensured jointly by application logic and integrity constraints.
Isolation	Concurrently executing transactions appear as if executed one after another (serially).	Handled by concurrency control (outside this chapter's scope).
Durability	Once a transaction commits, its changes persist in the database *even if the system crashes* immediately after.	Ensured by writing log records to stable storage before/at commit (redo on restart).

SHORTCUT TO REMEMBER

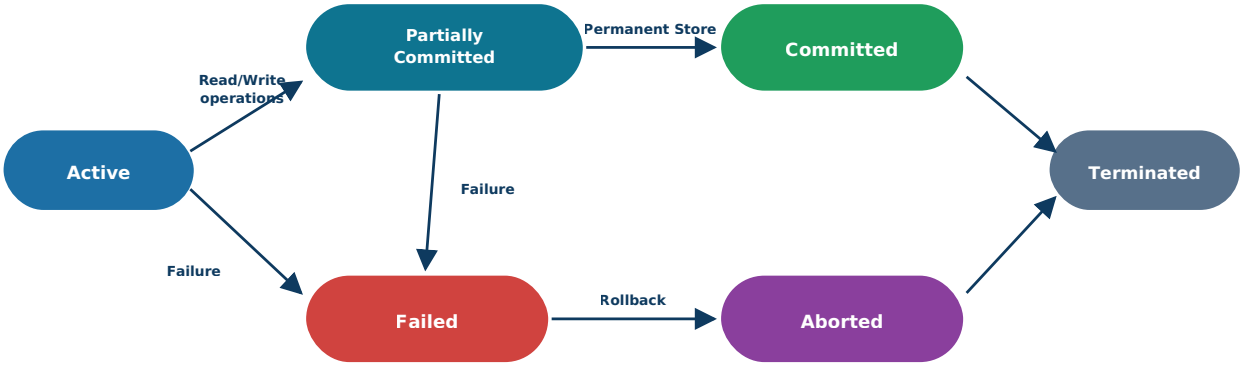
Remember ACID with: **A**ll-or-nothing, **C**orrectness, **I**solated execution, **D**urable once committed. Recovery mechanisms mainly guarantee **Atomicity** (via Undo) and **Durability** (via Redo).

2.3 States of a Transaction

During execution, a transaction passes through a well-defined sequence of states:

- **Active** — the initial state; the transaction is executing its operations.
- **Partially Committed** — after the final statement has executed, but before it is confirmed that changes will survive a possible failure (i.e., before the durability check).

- **Committed** — after successful completion; all changes have been made permanent in stable storage.
- **Failed** — when the normal execution cannot proceed further (due to hardware/logical error) and the transaction is halted.
- **Aborted** — after the transaction has been rolled back (undone) and the database has been restored to the state prior to the start of the transaction.
- **Terminated** — the transaction leaves the system, whether it committed or aborted.



Active → Partially Committed → Committed → Terminated (success path); Active/Partially Committed → Failed → Aborted → Terminated (failure path)

Fig 9.2 — Transaction state transition diagram.

> **EXAM POINT**
>
> The state just **before** "Committed" is **"Partially Committed"** — this is a favourite one-mark MCQ. It occurs after the last statement executes but before the system guarantees durability.

> **COMMON MISTAKE**
>
> Do not confuse **Failed** and **Aborted**. "Failed" means execution has stopped abnormally; "Aborted" means the system has already **rolled back / undone** the transaction's effects after it failed.

9.3 Log-Based Recovery

3.1 What is a Log?

A **log** is a sequence of records, kept on stable storage, that records every update made to the database by every transaction. Because it is on stable storage, the log **survives** a system crash even though main-memory buffers do not. This makes the log the single source of truth used to recover the database.

Standard Log Record Formats

Table 9.3 — Types of Log Records

Log Record	Meaning
`<Ti, start>`	Transaction Ti has started.
`<Ti, X, V_old, V_new>`	Ti has changed data item X from old value V_old to new value V_new.
`<Ti, commit>`	Ti has committed.
`<Ti, abort>`	Ti has aborted.

3.2 Write-Ahead Logging (WAL) Rule

The **Write-Ahead Log (WAL) rule** is the golden rule of log-based recovery:

> **WAL RULE**
> Before any change to the database on disk is made, the corresponding log record **must first** be written to stable storage. Also, before a transaction commits, **all** its log records must be written to stable storage.

This rule guarantees that if a crash occurs, the log always has enough information to know what each transaction did — even if the actual database write had not yet happened.

3.3 Deferred vs Immediate Database Modification

Table 9.4 — Deferred vs Immediate Modification

Aspect	Deferred Modification	Immediate Modification
When disk is updated	Only **after** the transaction partially commits (all writes are buffered/logged first).	Disk may be updated **while transaction is still active** (before commit).
Log content needed	Only new values (V_new) are strictly required.	Both old value (V_old) and new value (V_new) must be logged.
Undo needed?	Not needed — disk is never updated by an uncommitted transaction.	Needed — uncommitted changes may already be on disk.
Redo needed?	Needed — to redo committed transactions not yet reflected on disk.	Needed — same reason.

3.4 Recovery Procedure: Undo and Redo

After a crash, the recovery manager scans the log and builds two lists:

- **Undo-list** — transactions that have a `<Ti, start>` record but **no** `<Ti, commit>` record. These were incomplete, so all their changes must be **undone** (rolled back).
- **Redo-list** — transactions that have both `<Ti, start>` and `<Ti, commit>` records. These are committed, so their changes must be **redone** (reapplied) in case they had not yet reached the disk.

Recovery steps:

1. Scan the log from the end (or from last checkpoint) to build Undo-list and Redo-list.
2. **Undo** the operations of all transactions in the Undo-list, in reverse order, restoring old values.
3. **Redo** the operations of all transactions in the Redo-list, in the order they were logged, applying new values.

Worked Example

```
<T1, start>
<T1, A, 1000, 950>
<T1, B, 2000, 2050>
<T1, commit>
<T2, start>
<T2, C, 700, 600>
------------------- SYSTEM CRASH -------------------
// T1 is committed  -> REDO T1 (reapply A=950, B=2050)
// T2 is incomplete -> UNDO T2 (restore C=700)
```

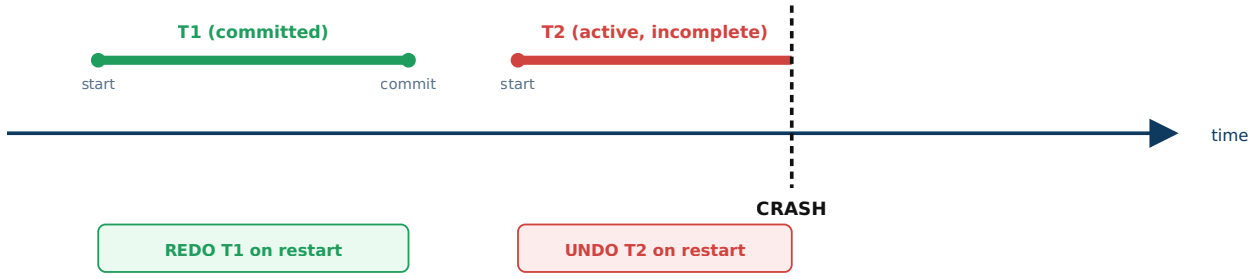


Fig 9.3 — Log-based recovery: committed transactions are REDONE, incomplete transactions are UNDONE.

> **REMEMBER**
>
> "**Undo the Uncommitted, Redo the Recorded-as-committed**." Only the presence of the `<Ti, commit>` record in the log decides whether a transaction is redone or undone — not whether its data physically reached the disk.

9.4 Checkpoint

4.1 What is a Checkpoint?

A **checkpoint** is a periodic "safe point" recorded by the DBMS in the log, marking that all transactions completed before this point are safely and permanently written to disk. It acts as a bookmark so that, during recovery after a crash, the system does not need to scan the log from the very beginning — it only needs to examine the log **from the most recent checkpoint onward**. This makes recovery much faster and reduces unnecessary Undo/Redo work.

4.2 Why Checkpoints Are Needed

In plain log-based recovery, after a crash the system must scan the **entire log from the very beginning** to find the Undo-list and Redo-list. As the database runs for a long time, the log grows huge, making two things very costly:

- Searching the whole log takes a long time.
- Many transactions in the log are already safely committed and reflected on disk, so redoing them again is wasted work.

A **checkpoint** solves this by periodically recording a "safe point" so recovery only needs to look at the log **after** the checkpoint.

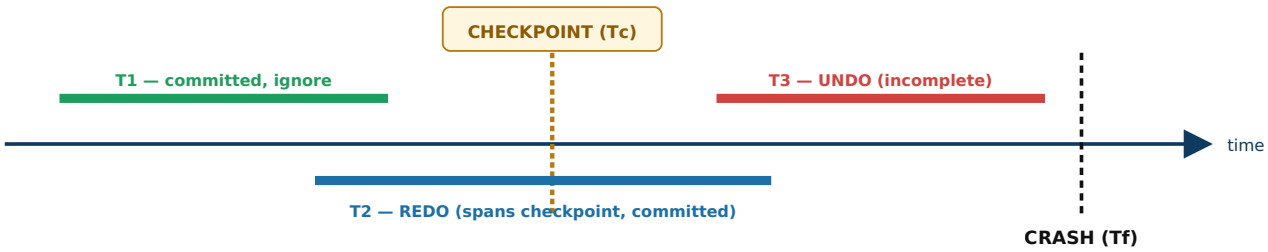
4.3 Checkpoint Procedure

1. Temporarily stop accepting new transactions (or note the set of currently active transactions).
2. Write all log records currently in memory (log buffer) to stable storage.
3. Write all modified/buffered database blocks from memory to disk.
4. Write a special `<checkpoint L>` record to the log, where *L* is the list of transactions active at checkpoint time.
5. Resume normal transaction processing.

4.4 Recovery Using a Checkpoint

On restart after a crash, the system:

1. Scans the log **backwards from the end** to find the **most recent** `<checkpoint L>` record.
2. Only transactions that were active at or started after this checkpoint need to be considered — everything before it is guaranteed safely on disk.
3. Builds Undo-list and Redo-list only from this reduced portion of the log, and performs Undo/Redo as usual.



Only T2 and T3 (active at/after checkpoint) are examined during recovery — T1 is skipped entirely.

Fig 9.4 — Checkpoint reduces recovery work: only transactions active at or after the checkpoint are checked.

EXAM POINT

With checkpointing, recovery needs to examine log records only from the **most recent checkpoint onward** — never the entire log. This is the key benefit tested in exams.

SHORTCUT

Checkpoint = "Save progress and mark a safe restart line." Anything fully finished before the checkpoint line can be safely ignored during recovery.

9.5 Shadow Paging

5.1 Concept

Shadow paging is a recovery technique that provides atomicity and durability **without maintaining a log** of database updates. Instead, it uses **two page tables** during the life of a transaction:

- **Shadow Page Table (Current/Committed):** points to the database pages as they were before the transaction started — never modified during the transaction.
- **Current Page Table:** a working copy used by the active transaction to make updates.

5.2 How It Works

1. At the start of a transaction, the current page table is made identical to the shadow page table (they point to the same database pages).
2. Whenever the transaction wants to modify a page, the DBMS does **not** overwrite the original page. Instead, it copies the page to a **new location on disk**, updates the copy, and makes the **current page table** entry point to this new page. The shadow page table is left untouched, still pointing to the original page.
3. All further reads/writes by the transaction use the current page table.
4. **On commit:** the shadow page table pointer is atomically switched to become the current page table — a single pointer update makes all changes permanent at once.
5. **On abort/crash before commit:** the current page table (and any new pages it created) is simply discarded; the shadow page table (old, consistent version) is still intact and becomes the database again.

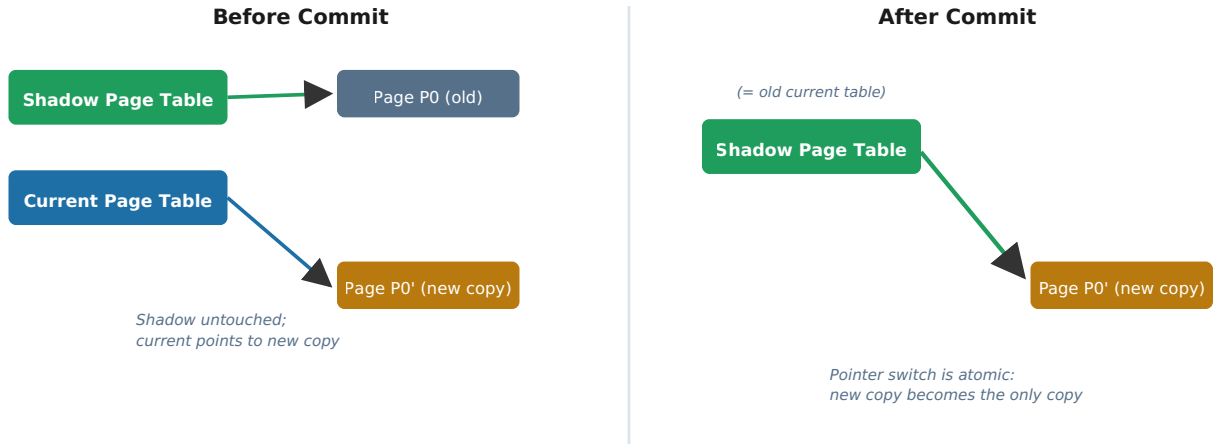


Fig 9.5 — Shadow paging: updates go to new page copies; commit atomically switches the shadow pointer.

5.3 Simple Worked Example

```
Initial: Shadow Page Table -> Page P (Balance = 1000)

Transaction T updates Balance to 1500:
  1. Copy P to new page P' on disk
  2. Current Page Table -> P' (Balance = 1500)
  3. Shadow Page Table still -> P (Balance = 1000)

If CRASH occurs here:
  -> Database uses Shadow Page Table -> P (Balance = 1000, safe, consistent)

If T COMMITS:
  -> Shadow Page Table pointer switched to P'
  -> Database now -> P' (Balance = 1500, permanent)
  -> Old page P can be reclaimed as free space
```

5.4 Advantages and Disadvantages

Table 9.5 — Shadow Paging: Pros and Cons

Advantages	Disadvantages
No need for Undo/Redo — no log required for recovery of updates.	Data gets **fragmented** across disk as new page copies are scattered.
Recovery is very fast — just discard the current table or keep the shadow table.	**Garbage collection** is required to reclaim old, unused pages after commit.
Commit is a single atomic pointer switch.	Difficult to support concurrent transactions (poor concurrency compared to log-based methods).
Simple concept, easy to reason about atomicity.	Page table itself can become large for big databases (deep/ multi-level tables costlier to copy).

COMMON MISTAKE

Students often say shadow paging "modifies the old page directly." It does **not** — the entire point is that the old (shadow) page is **never touched**; only a new copy is modified. This is what makes recovery log-free.

5.5 Log-Based Recovery vs Shadow Paging

Table 9.6 — Log-based Recovery vs Shadow Paging

Aspect	Log-Based Recovery	Shadow Paging
Extra structure needed	Log file on stable storage	Two page tables (shadow + current)
Undo/Redo required?	Yes	No
Commit operation	Write commit record to log	Atomic switch of page table pointer
Concurrency support	Good — widely used in practice	Poor — harder with concurrent transactions
Disk space overhead	Growing log file (trimmed by checkpoints)	Fragmentation from multiple page copies
Used in modern DBMS?	Yes — standard technique (e.g., ARIES)	Rarely used alone; some ideas reused in copy-on-write file systems

⚡ Quick Revision / Cheat Sheet — Chapter 9: Recovery Mechanisms

Storage & Failures

- **Volatile storage:** RAM, cache, registers — lost on power failure.
- **Non-volatile storage:** disk, SSD, tape — survives power loss, but can suffer disk/head crash.
- **Stable storage:** idealised failure-proof storage, approximated using mirrored/redundant disks.
- **3 failure types:** Transaction failure, System crash, Disk failure.

Transaction — Definition, ACID, States

- **Transaction:** a logical unit of work executed as an all-or-nothing operation.
- **ACID:** Atomicity, Consistency, Isolation, Durability.
- **6 states:** Active → Partially Committed → Committed → Terminated (success path); Active/Partially Committed → Failed → Aborted → Terminated (failure path).
- Recovery manager mainly enforces **Atomicity** (Undo) and **Durability** (Redo).

Log-Based Recovery

- **Log record:** <Ti, start>, <Ti, X, old, new>, <Ti, commit>, <Ti, abort>.
- **WAL rule:** log record written to stable storage *before* the corresponding database write, and before commit.
- **Deferred modification:** disk updated only after commit → no Undo needed, only Redo.
- **Immediate modification:** disk may update before commit → both Undo and Redo needed.
- **Recovery rule:** committed transactions → REDO; incomplete transactions → UNDO.

Checkpoint

- **Purpose:** avoid scanning the entire log; reduce recovery time.
- **Checkpoint record:** <checkpoint L> — L = list of active transactions at that moment.
- **Recovery:** scan log backward to find the latest checkpoint; only process transactions active at/after it.

Shadow Paging

- **Two page tables:** Shadow (old, safe) and Current (working copy).
- Updates create a **new page copy**; original page is never modified.
- **Commit:** atomic switch of shadow pointer to current table.
- **No log, no Undo/Redo needed** — but causes fragmentation and needs garbage collection.
- Poor support for concurrent transactions compared to log-based recovery.

One-Line Exam Facts

- Recovery is mainly needed because volatile storage (RAM) loses data on a system crash.
- Stable storage is implemented using mirrored non-volatile storage, not a real physical device.
- State immediately before "Committed" = "Partially Committed."
- WAL = log first, data write later — the foundation rule of log-based recovery.
- Deferred modification needs only Redo; Immediate modification needs both Undo and Redo.
- Checkpoint lets recovery ignore everything before the last checkpoint.
- Shadow paging achieves atomicity via a single pointer switch, not via logging.